

78. Write a C program to stimulate
- Producers-Consumers problem using semaphores.
 - Dining-Philosopher problem.

```
> #include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
#define N 5

/* Producer Consumer */

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = i;
        printf("Produced: %d at buffer [%d]\n", i, in);
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}
```

```

void *consumer (void *arg) {
    for (int i = 0; i < 10; i++) {
        usleep(200000);
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item,
            out);
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

```

```

void run_producer_consumer() {

```

```

    pthread_t p, c;

```

```

    sem_init(&empty, 0, BUFFER_SIZE);

```

```

    sem_init(&full, 0, 0);

```

```

    sem_init(&mutex, 0, 1);

```

```

    pthread_create(&p, NULL, producer, NULL);

```

```

    pthread_create(&c, NULL, consumer, NULL);

```

```

    pthread_join(p, NULL);

```

```

    pthread_join(c, NULL);

```

```

    sem_destroy(&empty);

```

```

    sem_destroy(&full);

```

```

    sem_destroy(&mutex);

```

```

}

```


/* Dining philosophers */

sem_t chopstick[N];

sem_t room;

void *philosopher(void *num){

int id = *(int *) num;

printf("Philosopher %d is thinking.\n", id);

sleep(1);

sem_wait(&room);

printf("Philosopher %d is picked up left fork
%d.\n", id, id);

sem_wait(&chopstick[id]);

printf("Philosopher %d picked up right fork %d.\n",
id, (id+1) % N);

sem_wait(&chopstick[(id+1) % N]);

printf("Philosopher %d is eating.\n", id);

sleep(2);

printf("Philosopher %d put down forks %d and %d.\n",
id, id, (id+1) % N);

sem_post(&chopstick[id]);

sem_post(&chopstick[(id+1) % N]);

sem_post(&room);

return NULL;

}

```

void run_dining_philosophers() {
    pthread_t ph[N];
    int ids[N];
    sem_init(&room, 0, N-1);
    for (int i=0; i<N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (int i=0; i<N; i++)
        ids[i] = i;
    pthread_create(&ph[i], NULL, philosopher,
                  &ids[i]);
    usleep(150000);
}

for (int i=0; i<N; i++)
    pthread_join(ph[i], NULL);
for (int i=0; i<N; i++)
    sem_destroy(&chopstick[i]);
sem_destroy(&room);
}

int main() {
    int choice;

    while(1) {
        printf("In --- MENU --- In");
        printf("1. Producer - Consumer Problem\n");
        printf("2. Dining Philosophers Problem\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
}

```



```
switch (choice) {
```

```
case 1:
```

```
printf("In -- Producer-Consumer  
output -- \n");
```

```
run-producer-consumer();
```

```
break;
```

```
case 2:
```

```
printf("In -- Dining Philosopher output  
run-dining-philosopher();
```

```
break;
```

```
case 3:
```

```
printf("Exiting -- \n");
```

```
exit(0);
```

```
default:
```

```
printf("Invalid choice");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

output:

--MENU--

1. Producer-Consumer Problem
2. Dining Philosophers Problem
3. Exit

Enter your choice: 1

-- Producer-Consumer Output --

Produced: 0 at buffer [0]

Produced: 1 at buffer [1]

Consumed: 0 from buffer [0]

Produced: 2 at buffer [2]

Produced: 3 at buffer [3]

Consumed: 1 from buffer [1]

Produced: 4 at buffer [4]

Consumed: 2 from buffer [2]

Consumed: 3 from buffer [3]

Consumed: 4 from buffer [4]

--MENU--

1. Producer-Consumer Problem
2. Dining Philosophers Problem
3. Exit

Enter your choice: 2

-- Dining Philosophers Output --

philosopher 0 is thinking

philosopher 1 is thinking

philosopher 2 is thinking

philosopher 3 is thinking

philosopher 4 is thinking

Philosopher 0 picked up left fork 0.

Philosopher 0 picked up right fork 1.

Philosopher 0 is eating.

Philosopher 1 picked up left fork 1.

Philosopher 2 picked up left fork 2.

Philosopher 2 picked up right fork 3.

Philosopher 2 is eating.

Philosopher 3 picked up left fork 3.

Philosopher 0 put down forks 0 and 1.

Philosopher 1 picked up right fork 2.

Philosopher 4 picked up left fork 4.

Philosopher 4 picked up right fork 0.

Philosopher 4 is eating.

Philosopher 2 put down forks 2 and 3.

Philosopher 1 is eating.

Philosopher 3 picked up right fork 4.

Philosopher 4 put down fork 4 and 0.

Philosopher 3 is eating.

Philosopher 1 put down forks 1 and 2.

Philosopher 3 put down fork 3 and 4.

--MENU--

1. Produce & Consumer Problem

2. Dining Philosopher Problem

3. Exit

enter your choice: 3

Exiting.....